

***RAGMED*: SISTEMA RAG PARA DIAGNÓSTICO DE ENFERMEDADES**

**SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA**

Viren S. Dhanwani Dhanwani

Universidad Nacional de Educación a Distancia

Máster Universitario en Ingeniería Informática

2026-05-22

**ÍNDICE**

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Introducción</b>                               | <b>2</b> |
| <b>2. Crawler</b>                                    | <b>2</b> |
| <b>3. Módulo de Recuperación de Información (IR)</b> | <b>5</b> |
| <b>4. Evaluación</b>                                 | <b>7</b> |
| <b>5. Experimentos adicionales</b>                   | <b>9</b> |
| <b>6. Conclusiones</b>                               | <b>9</b> |
| <b>7. Referencias</b>                                | <b>9</b> |

# SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

## 1. Introducción

En este documento se presenta la práctica de la asignatura “SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA”, del “MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA” de la UNED.

El objetivo de la práctica es diseñar un sistema RAG [1] sobre una temática de libre elección con tres módulos: *crawler*, sistema de recuperación de información y generación de texto. Como punto de partida, se proporcionó un sistema de referencia sobre información de Pokémon extraída de *WikiDex* [2].

El dominio escogido es el médico, implementando el sistema *RAGMED*. El objetivo es que un profesional de la salud obtenga una lista de posibles enfermedades a partir de los síntomas que describe una persona usando lenguaje natural. El código está escrito en *Python*.

## 2. Crawler

Un *crawler web* es un programa automatizado que recorre páginas de internet siguiendo enlaces, y extrae su contenido para construir un *corpus* de texto [3]. En el contexto de un sistema RAG, el *crawler* constituye la etapa de adquisición de datos.

Para *RAGMED* en el fichero *Sistema/ragmed\_crawler.py* se implementó la clase *RADGMED\_crawler*. El *crawler* cubre dos etapas: la descarga de la lista de enfermedades y la descarga del contenido individual de cada artículo. En la primera etapa, el *crawler* recorre las páginas de índice de la fuente de datos y extrae los nombres de enfermedades enlazadas, guardándolos en el fichero *disease\_list.txt*. En la segunda etapa, para cada nombre de la lista se

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

consulta la fuente de datos para obtener el extracto del texto del artículo, lo segmenta en cuatro secciones y escribe el bloque resultando en *diseases.txt*.

La fuente de datos elegida es la *Wikipedia* en inglés, accedida a través de su API REST oficial: *MediaWiki* API [4]. Con el fin de desacoplar el *crawler* a la fuente de datos utilizada (para reemplazarla fácilmente), en el constructor `__init__` de *RAGMED\_crawler* se inyecta como dependencia un objeto de tipo *DiseaseSource*, que en esta práctica solo tiene una implementación: *WikipediaDiseaseSource* con dos métodos: *list\_diseases* para listar las enfermedades y *fetch\_extract* para obtener la información de una enfermedad concreta.

La decisión de utilizar la API en lugar de *scraping* directo del HTML se tomó con el fin de simplificar el flujo, ya que la API ofrece parámetros como *explaintext=true* para que el servidor devuelva el artículo en texto plano, con los encabezados marcados mediante la convención “== Nombre de sección ==”. Esto elimina la necesidad de eliminar manualmente elementos HTML, además de cumplir con los términos de *Wikipedia* para uso académico.

La URL del *endpoint* utilizado es la siguiente:

None

```
https://en.wikipedia.org/w/api.php?action=query&prop=extracts&explaintext=true&titles={nombre_enfermedad}&format=json&redirects=1
```

No obstante, la API no ofrece un *endpoint* para listar artículos de una misma categoría de manera cómoda, por lo que el *crawler* recorre las páginas de *Wikipedia* de enfermedades por índice alfabético, mediante *scraping* HTML con *BeautifulSoup* [5]. De cada página se extraen los enlaces a artículos individuales (`<li>` con `<a href="/wiki/...">`), filtrando aquellos que

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

contienen dos puntos en la ruta (páginas especiales) y los que no apuntan al espacio de nombres principal de *Wikipedia*, teniendo que eliminar manualmente elementos HTML que generan ruido.

Para generar el *corpus*, se extrae de cada artículo de enfermedad cuatro fragmentos:

- *Lead* (párrafo inicial). Texto resumen que va antes del primer encabezado.
- *Signs and symptoms* (signos y síntomas). La sección que más nos interesa.
- *Causes* (causas). Complementa a la sección anterior.
- *Treatment / Management / Prevention* (tratamiento). Distintos artículos emplean denominaciones diferentes para la misma sección temática.

El *crawler* utiliza los métodos auxiliares *parse\_sections* y *find\_section* del fichero *\_helpers.py* para extraer las secciones (mediante la expresión regular *\_H2\_PATTERN*) y recorrer una sección específica, respectivamente.

El *corpus* final se almacena en el fichero *diseases.txt*, donde se recoge cada enfermedad con su resumen, síntomas, causas y tratamiento, en un formato unificado. Si una sección no está disponible, se escribe el literal (*No information available.*). Además, el *crawler* genera dos ficheros intermedios por enfermedad: *diseases/{nombre}.txt* y *diseases/{nombre}\_clean.txt* para facilitar la depuración y la reanudación de ejecuciones interrumpidas.

El principal problema encontrado es la limitación de tasa de peticiones, con respuesta HTTP 429, debido a que *Wikipedia* impone límites de velocidad. Cuando la petición al servidor falla, el *crawler* implementa una estrategia de reintentos (3 reintentos, cada uno tras 2<sup>nº</sup> intento segundos). Adicionalmente, entre cada petición se introduce una espera fija de 1,5 segundos.

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

Al ejecutar el *crawler* sobre el índice alfabético completo, se identificaron 5.390 nombres de enfermedades en las páginas del índice, descargando con éxito 4.312 de ellas. La calidad varía notablemente: hay enfermedades bien documentadas con todas sus secciones mientras que otras solo tienen párrafo inicial y no cuentan con campo de síntomas y causas (*Aagenaes syndrome*).

### 3. Módulo de Recuperación de Información (IR)

El módulo de Recuperación de Información determina qué fragmentos del *corpus* se proporcionan al modelo de lenguaje como contexto. En *RAGMED*, se implementa en la clase *RADGMED\_rag* del fichero *Sistema/ragmed\_rag.py*, y extiende el sistema de referencia con tres mejoras: estrategia de fragmentación semántica, conjunto ampliado de funciones de similitud y un modo de recuperación híbrida que combina distintos métodos.

La clase de referencia implementa recuperación densa mediante embeddings *bge-base-en-v1.5-gguf* con similitud coseno. Esto permite capturar equivalencias semánticas automáticamente (*shortness of breath*  $\rightarrow$  *dyspnea*), pero no discrimina por frecuencia del término y puede penalizar términos clínicos poco frecuentes en el *corpus* de preentrenamiento.

La principal mejora es la introducción de recuperación híbrida BM25 + coseno. BM25 destaca cuando la consulta contiene terminología clínica exacta, gracias a su IDF y saturación de frecuencia de término. La función de puntuación híbrida implementada en *HybridSimilarity* combina ambas señales mediante una suma ponderada:

$$\text{score\_hybrid}(Q, D) = \alpha \cdot \text{coseno\_emb}(Q, D) + (1 - \alpha) \cdot \text{BM25\_norm}(Q, D)$$

donde  $\alpha \in [0, 1]$  es el parámetro de mezcla (valor por defecto: 0,5), *coseno\_emb* es la similitud coseno entre los embeddings de la consulta y el fragmento (rango  $[0, 1]$  para

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

embeddings no negativos), y *BM25\_norm* es la puntuación BM25 normalizada por el máximo del lote para llevarla al intervalo  $[0, 1]$ :

$$\text{BM25\_norm}(Q, D) = \frac{\text{BM25}(Q, D)}{\max_{D' \in \mathcal{C}} \text{BM25}(Q, D') + \varepsilon}$$

con  $\varepsilon = 10^{-8}$  para evitar la división por cero. La normalización por máximo preserva el ordenamiento relativo de las puntuaciones BM25 dentro del lote y las hace directamente comparables a las similitudes coseno, que ya están acotadas en  $[0, 1]$ .

La implementación utiliza la variante *BM25Okapi* de la biblioteca *rank\_bm25* [6], con parámetros por defecto  $k1=1.5$ ,  $b=0.75$ . La *tokenización* del *corpus* y de la consulta se realiza de forma consistente: conversión a minúsculas y separación por espacios. El índice BM25 se construye una sola vez en el método *load\_dataset*, junto con el proceso de embedding del corpus, y queda disponible en memoria para todas las consultas posteriores.

*RADGMED\_rag* implementa cuatro funciones de similitud individuales y una híbrida, seleccionables mediante el parámetro *--similarity-fn* de la interfaz de línea de comandos. La Tabla 1 resume sus características y su adecuación al dominio médico, incluyendo opciones como euclidiana o jaccard con fines comparativos.

En cuanto a la estrategia de *chunking*, *RAGMED\_rag* implementa una estrategia de fragmentación a nivel de sección. El método *\_parse\_chunks* analiza el fichero *diseases.txt* e

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

identifica los bloques de enfermedad por su patrón de encabezado. A continuación, `_extract_disease_chunks` genera hasta cuatro fragmentos por enfermedad, una por cada sección.

Tabla 1: Funciones de similitud implementadas en RAGMED

| Función               | Tipo       | Descripción Breve                          | Idoneidad para RAGMED    |
|-----------------------|------------|--------------------------------------------|--------------------------|
| Coseno (denso)        | Densa ▾    | Ángulo entre vectores de <i>embedding</i>  | Buena (maneja sinónimos) |
| BM25 (Okapi)          | Dispersa ▾ | TF saturado + IDF + normaliz. longitud     | Buena (términos exactos) |
| Euclidiana            | Densa ▾    | $1/(1+\text{distancia L2})$                | Moderada                 |
| Jaccard               | Dispersa ▾ | Solapamiento de conjuntos de <i>tokens</i> | Débil                    |
| Híbrida (BM25+coseno) | Mixta ▾    | Suma ponderada                             | Óptima                   |

La fragmentación por sección mejora la precisión: el fragmento de *Signs and Symptoms* contiene solo el texto relevante para la consulta, sin ruido etiológico ni terapéutico. Los fragmentos vacíos se descartan para no consumir posiciones en el *ranking*. Los parámetros principales del sistema se recogen en la tabla 2.

### 4. Evaluación

La evaluación de *RAGMED* se realiza mediante el diseño de diez preguntas de ejemplo con respuesta de referencia definida previamente. Cada pregunta se formula en inglés, en primera persona y desde la perspectiva de un paciente que describe sus síntomas. Para cada pregunta, se identifica manualmente la enfermedad esperada y se localiza el pasaje del *corpus* correspondiente.

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

Tabla 2: Parámetros de configuración

| Parámetro           | Valor por defecto          | Descripción                      |
|---------------------|----------------------------|----------------------------------|
| similarity_fn       | hybrid                     | Función de recuperación activa   |
| alpha               | 0,5                        | Peso del coseno en modo híbrido  |
| top_n               | 5                          | Número de fragmentos recuperados |
| Modelo de embedding | bge-base-en-v1.5-gguf      | Embeddings de 768 dimensiones    |
| Modelo de lenguaje  | Llama-3.2-1B-Instruct-GGUF | Generación de respuestas         |

Se evaluó el sistema con los valores por defecto (véase Tabla 2), con tres métricas independientes: precisión (respuesta relevante), cobertura (cubre los síntomas) y veracidad (no alucina), y cada métrica se evalúa con una puntuación de 0, 0,5 o 1.

Las diez preguntas cubren diversas categorías diagnósticas (viral, infección respiratoria, bacteriana...), con el fin de abordar distintos dominios médicos. Las preguntas con el correspondiente pasaje relevante en el *corpus* se encuentran en el fichero *Evaluación/preguntas\_ground\_truth.md*.

El resultado obtenido tras ejecutar el *script* de evaluación (*Sistema/ragmed\_eval*) da un total de 17,5/30, con una distribución bimodal: tres preguntas obtuvieron la máxima nota y tres obtuvieron una puntuación casi nula. En este análisis se ha podido observar que el sistema funciona de forma óptima cuando los síntomas de la consulta coinciden léxicamente con el *corpus*. No obstante, se detectaron patrones de fallo como la brecha léxica (“*feeling cold all the*



## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

*time*” no caza con el síntoma “*poor ability to tolerate cold*”. Además, en enfermedades como la malaria o tuberculosis los síntomas son recurrentes en otras enfermedades, las cuales se priorizan aunque no tengan esa combinación exacta de síntomas. Por último, cabe destacar que por la naturaleza sensible de los diagnósticos, el modelo en ocasiones se negaba a contestar, instando a que el usuario se comuniqué con un experto sanitario, aún indicando al modelo que es un asistente médico y que responda según el contexto facilitado, especialmente para enfermedades más sensibles como la malaria.

Para mejorar estos resultados, aumentar *top\_n* de 5 a 10 fragmentos podría reducir los fallos en enfermedades con síntomas genéricos, y la aplicación de *query expansion* mejoraría el componente BM25 de la recuperación al introducir sinónimos para determinados síntomas.

### 5. Experimentos adicionales

Adicionalmente, se realizaron dos experimentos: comparación del modelo generativo base con un modelo de mayor tamaño y comparación de tres tamaños de contexto por fragmento recuperado. En el primer experimento, se comparó el modelo base *Llama-3.2-1B-Instruct-GGUF* con las mismas preguntas, obteniendo una mejora de puntuación notable (22/30 vs 17,5/30), aumentando 4,5 puntos. Los fallos mencionados anteriormente respecto a la malaria y la tuberculosis siguen presentes en este modelo debido a que son fallos de recuperación.

Por otro lado, en el segundo experimento se introdujo el parámetro *max\_context\_chars*, el cual controla cuántos caracteres de cada fragmento recuperado se incluyen en el *prompt* del LLM. Se compararon tres valores: 500, 1.500 y 3.000, obteniendo puntuaciones similares para los 3 casos (17,5, 17,5 y 15,5 respectivamente). Así pues, se puede concluir que más contexto no implica mejores respuestas.

### 6. Conclusiones y líneas futuras

Se ha diseñado e implementado el sistema *RAGMED* que, dado un conjunto de síntomas descritos en lenguaje natural, devuelve un diagnóstico fundamentado en un *corpus* de artículos de *Wikipedia* sobre enfermedades. Dispone de versión por línea de comandos parametrizada (*Sistema/ragmed\_main.py*) y de versión *web* (*Sistema/ragmed\_web.py*).

El *crawler* implementado genera un *corpus* con una gran variedad de enfermedades, y el procesamiento previo fue necesario para estructurar la información y facilitar la obtención de resultados al módulo de recuperación de información. Se podría mejorar la lista de enfermedades utilizando una fuente de datos especializada en el dominio médico (*Medline*, *OMS*...).

Para el módulo de recuperación, la función de recuperación híbrida otorga resultados aceptables, obteniendo puntuación alta en varias de las preguntas de evaluación. Posteriormente, se obtuvieron mejores resultados empleando un modelo de mayor tamaño.

### 7. Referencias

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] WikiDex contributors, "WikiDex," WikiDex, la enciclopedia Pokémon. [Online]. Available: <https://www.wikidex.net/wiki/WikiDex>. [Accessed: May 2026].
- [3] W. B. Croft, D. Metzler, and T. Strohman, "Search Engines: Information Retrieval in Practice", Pearson Education, 2015.
- [4] Wikimedia Foundation, "MediaWiki API," Wikipedia. [Online]. Available: [https://www.mediawiki.org/wiki/API:Main\\_page](https://www.mediawiki.org/wiki/API:Main_page). [Accessed: May 2026].

## SISTEMAS DE INFORMACIÓN NO ESTRUCTURADA

- [5] L. Richardson, "Beautiful Soup Documentation," Crummy.com. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. [Accessed: May 2026].
- [6] D. Brown, "rank-bm25: A two-line search engine," GitHub. [Online]. Available: [https://github.com/dorianbrown/rank\\_bm25](https://github.com/dorianbrown/rank_bm25). [Accessed: May 2026].